

CS184/284A Fall 2026 Homework 1 Write-Up

Names: Dongyang Dai and Zihan Wang

Webpage: cal-cs184-student.github.io/hw1-prince-writeup/

Repository: github.com/cal-cs184-student/hw1-prince

Overview

In this assignment, we built the main stages of a rasterization pipeline for simplified SVG scenes. We began with single-color triangle coverage, then extended the rasterizer with supersampling for antialiasing. We implemented homogeneous transforms for hierarchical scene placement, used barycentric coordinates to interpolate color and texture coordinates, and added nearest-neighbor and bilinear texture filtering. Finally, we used mipmaps and level-of-detail selection to reduce texture aliasing during minification. Together, these components produce a functional vector graphics renderer and show how sampling choices affect both image quality and computational cost.

Task 1: Drawing Single-Color Triangles

Bounding box **Edge functions** **4×4 block optimization**

Rasterization. We compute the triangle's axis-aligned bounding box, clamp it to the framebuffer, and evaluate coverage at each pixel center $(x + 0.5, y + 0.5)$. Three signed edge functions determine whether the sample lies inside the triangle, including samples exactly on an edge.

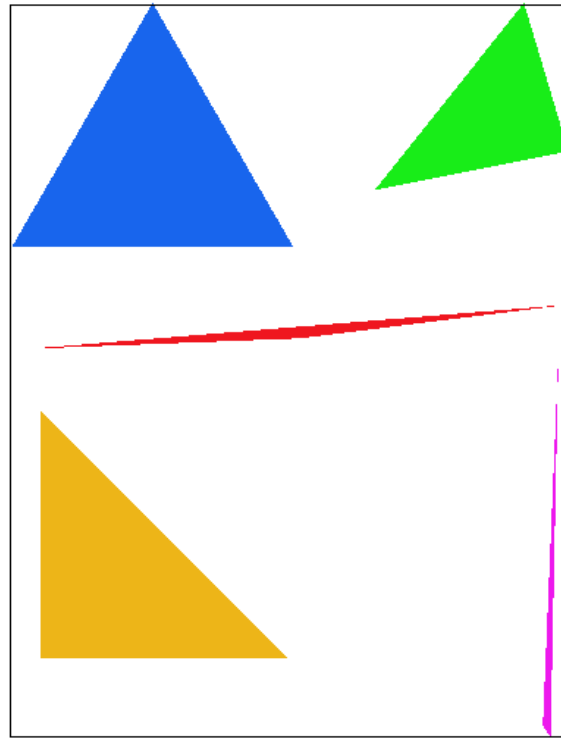


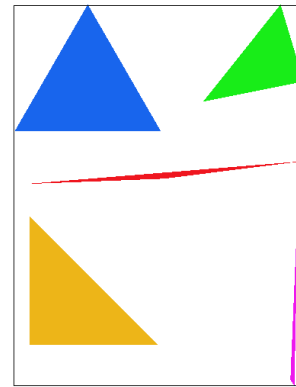
Figure 1.1: basic/test4.svg under the default view with the pixel inspector enabled

▼ Supplementary Task 1 validation renders

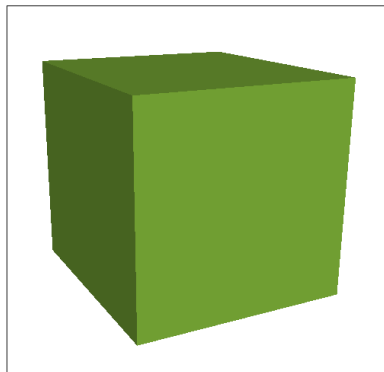
These four images are additional sanity checks and are not required by the base Task 1 rubric. Keep them small so the required test4.svg result above remains the focus.



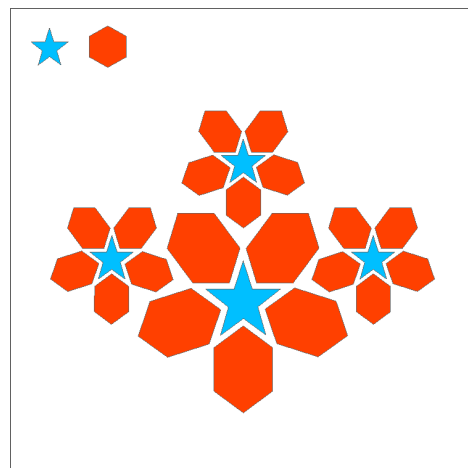
basic/test3.svg



basic/test4.svg



basic/test5.svg



basic/test6.svg

Extra Credit — Make your triangle rasterizer super fast. Inside the bounding box, we process 4-by-4 blocks. A block whose four corners are inside can be filled immediately; a block whose four corners are outside the same edge can be rejected. Only boundary blocks require per-pixel tests. The baseline still visits no samples outside the triangle's bounding box, so it is no worse than a standard bounding-box traversal.

```
int RasterizerImp::classify_edge(float ax, float ay,
                                float bx, float by,
                                float left, float top,
                                float right, float bottom,
                                float dir) {

    float dx = (bx - ax) * dir;
    float dy = (by - ay) * dir;

    float corner_0 = -(left - ax) * dy + (top - ay) * dx;
    float corner_1 = -(left - ax) * dy + (bottom - ay) * dx;
    float corner_2 = -(right - ax) * dy + (top - ay) * dx;
```

```

float corner_3 = -(right - ax) * dy + (bottom - ay) * dx;

float corner_max = max({corner_0, corner_1, corner_2, corner_3});
float corner_min = min({corner_0, corner_1, corner_2, corner_3});

if (corner_max < 0) return -1;
if (corner_min >= 0) return 1;
return 0;
}

```

How the block test works. We first compute the signed triangle area and use `dir` to normalize clockwise and counterclockwise triangles to the same inside convention. The bounding box is then divided into 4-by-4 blocks. For each triangle edge, `classify_edge()` evaluates the edge function only at the block's four extreme sample centers. A negative maximum means the entire block is outside that edge, so the block can be rejected immediately. A nonnegative minimum means the entire block is inside that edge, while a mixed sign identifies a boundary block.

Why it is faster. Blocks classified inside all three edges can be filled without repeating three coverage tests for every sample, and blocks outside any one edge are skipped as soon as that edge is found. Per-sample edge tests are reserved for mixed blocks near the triangle boundary. Large triangles therefore spend most of their time in the cheap accept/reject paths, while the 4-by-4 size keeps the extra corner-test overhead modest for small or thin triangles.

Block-rasterization benchmark

We compared the baseline `draw_baseline` executable with the block-optimized `draw` executable. Each per-scene result is the average of 10 runs, measured as the total time spent in `rasterize_triangle()`.

Scene	Resolution	Baseline avg. (ms)	Block avg. (ms)	Speedup	Time reduction
test3.svg	332 × 347	2.173451	1.307930	1.662×	39.82%
test4.svg	488 × 643	0.290644	0.082443	3.525×	71.63%
test5.svg	356 × 345	0.425642	0.124638	3.415×	70.72%
test6.svg	1200 × 1200	2.742915	1.053889	2.603×	61.58%

Task 2: Antialiasing by Supersampling

Subpixel grid Sample buffer **Resolve by averaging**

Method. For sample rate n , each pixel is divided into a $\sqrt{n} \times \sqrt{n}$ grid. We run the same edge tests at every subpixel center and store the results in a buffer of size $width \times height \times sample_rate$.

Why it works. During `resolve_to_framebuffer()`, the subpixel colors are averaged. Partially covered pixels therefore become blends of foreground and background instead of binary decisions, which smooths thin triangle boundaries. Higher sample rates improve coverage estimates but increase work and sample-buffer memory roughly in proportion to the number of samples.



Figure 2.1: 1 sample per pixel (no supersampling)

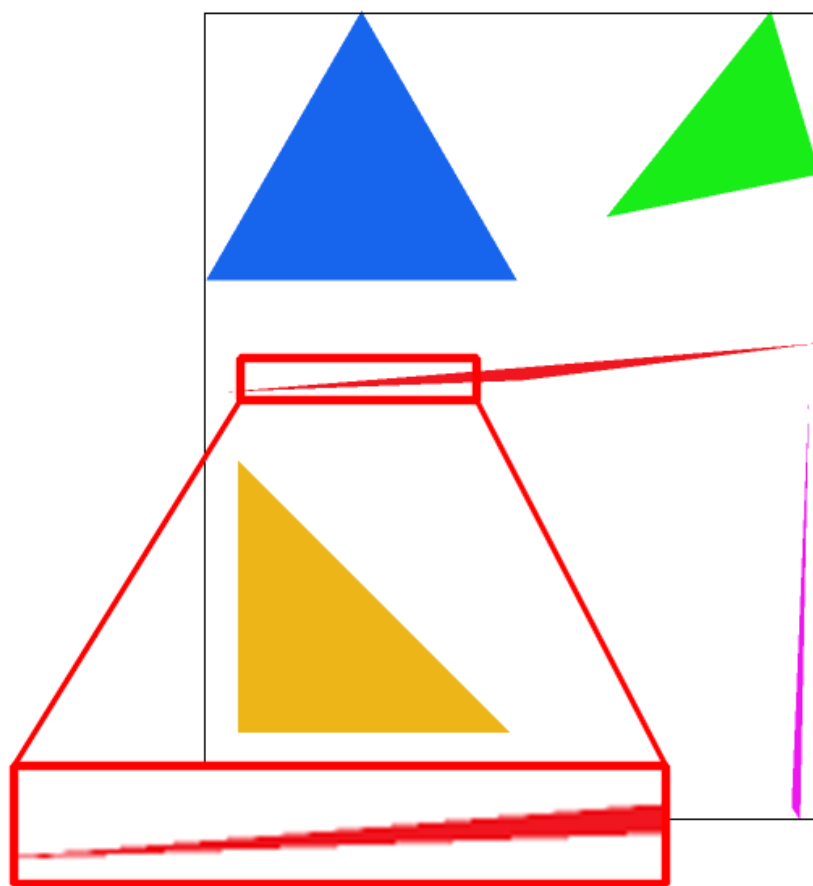


Figure 2.2: 4 samples per pixel

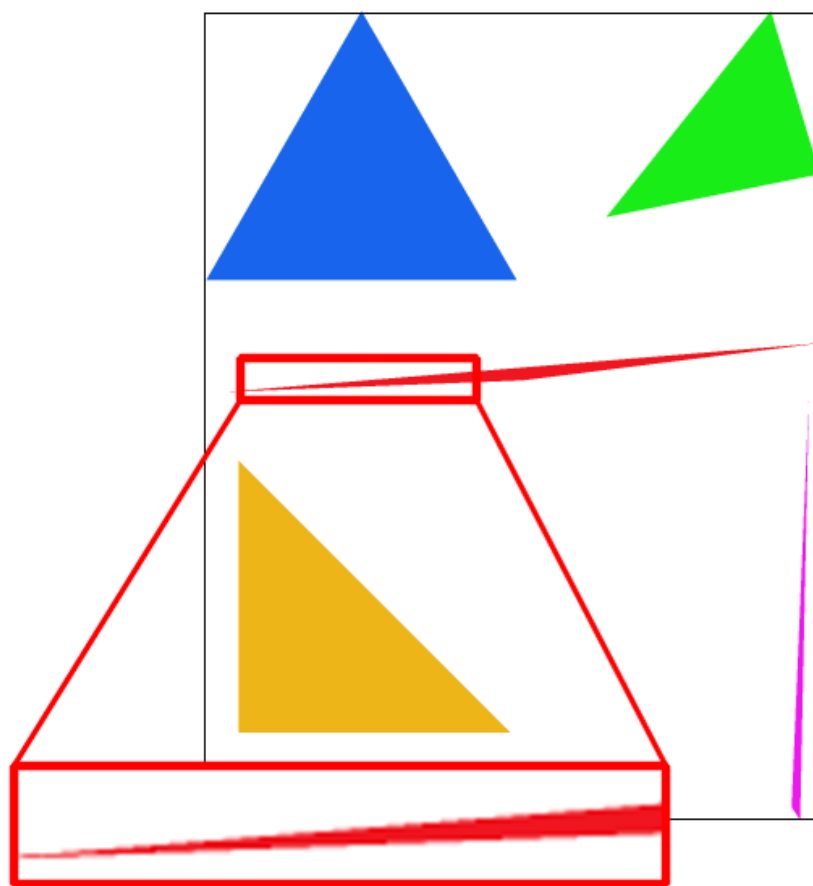


Figure 2.3: 9 samples per pixel

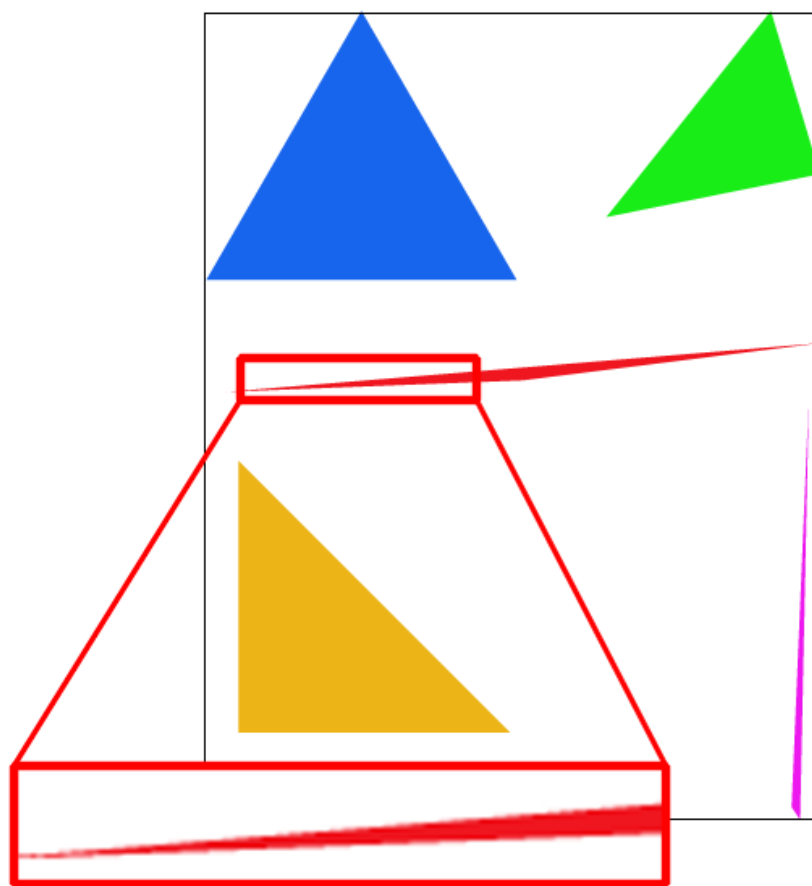


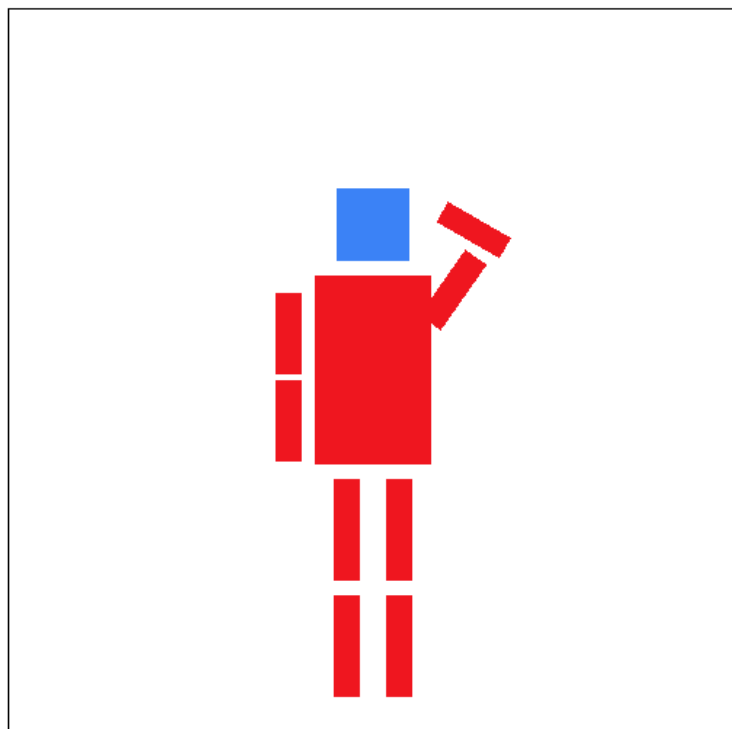
Figure 2.4: 16 samples per pixel

Task 3: Transforms

[Translate](#) [Scale](#) [Rotate](#) [Hierarchical transforms](#)

Implementation. Translation, scaling, and rotation are represented as 3-by-3 homogeneous matrices. Nested SVG groups compose local transforms with ancestor transforms, preserving the intended hierarchy.

Our robot. We changed Cubeman into a standing salute pose: the torso is scaled into a vertical rectangle, the legs remain straight, the left arm rests at the side, and the right arm uses nested translations and rotations to bend toward the temple. The blue head helps distinguish the transformed parts.

Figure 3.1: Robot rendered with SVG transforms. ([my_robot.svg](#))

Extra Credit — Interactive viewport rotation. We added keyboard-controlled rotation to the rendered view. The animation below demonstrates the viewport rotation directly in the original GUI.

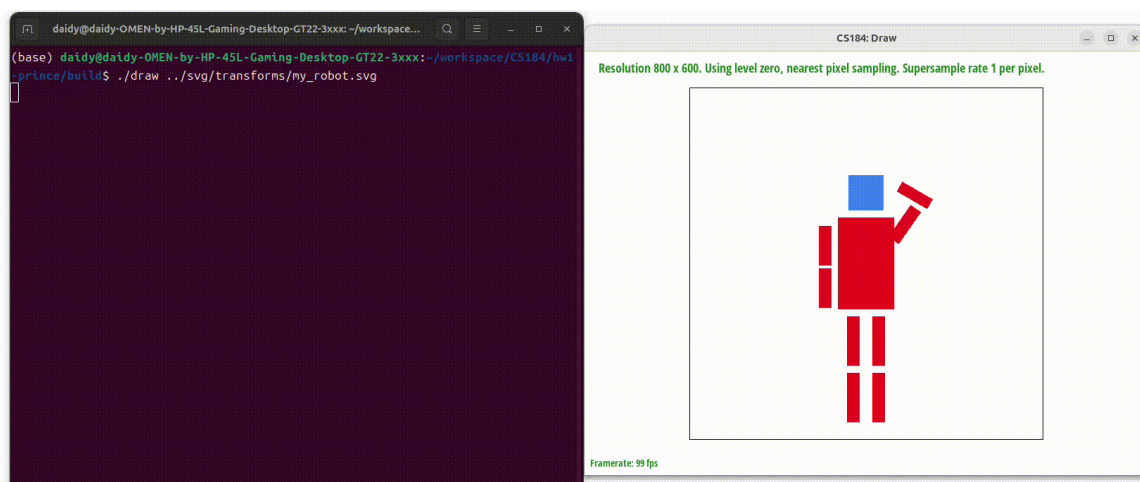


Figure 3.2: Keyboard-controlled rotation of the rendered scene

Rotation about the viewport center

The original transformation from SVG coordinates to screen coordinates can be written as

$$p_{\text{screen}} = M_{\text{ndc} \rightarrow \text{screen}} M_{\text{svg} \rightarrow \text{ndc}} p_{\text{svg}}.$$

Our NDC center is $C = (0.5, 0.5)$. To rotate the complete rendered view about this center, we compose three transformations:

1. Translate the center to the origin: $T(-0.5, -0.5)$.
2. Rotate about the origin: $R(\theta)$.
3. Translate the center back to its original position: $T(0.5, 0.5)$.

$$M_{\text{view rotation}} = T(0.5, 0.5) R(\theta) T(-0.5, -0.5) = \begin{bmatrix} c & -s & 0.5s - 0.5c + 0.5 \\ s & c & -0.5s - 0.5c + 0.5 \\ 0 & 0 & 1 \end{bmatrix}$$

Task 4: Barycentric Coordinates

α, β, γ Attribute interpolation Color gradients

Barycentric coordinates express a point relative to the three vertices of a triangle rather than relative to the global x- and y-axes. This makes them especially useful in rasterization: they describe not only where a sample lies inside a triangle, but also how strongly each vertex contributes to that sample.



For a triangle with vertices A , B , and C , any point P in the same plane can be written as $P = \alpha A + \beta B + \gamma C$, where $\alpha + \beta + \gamma = 1$. The values α , β , and γ are the barycentric coordinates of P . Each weight is 1 at its corresponding vertex and decreases linearly to 0 along the opposite edge. A point lies inside or on the triangle when all three weights are nonnegative, so the coordinates are useful for both coverage testing and interpolation.

In `rasterize_interpolated_color_triangle()`, we compute these three weights at each covered sample and use them to blend the per-vertex colors: $C(P) = \alpha C_A + \beta C_B + \gamma C_C$. In Figure 4.1, samples near one vertex are dominated by that vertex's red, green, or blue color; samples near the center receive comparable contributions from all three. Because the weights vary linearly across the triangle, the resulting gradient is continuous. The same operation later lets us interpolate texture coordinates across textured triangles.

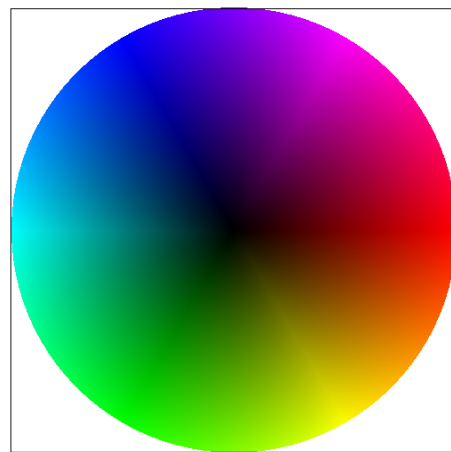


Figure 4.2: Rendered result using barycentric color interpolation

Task 5: "Pixel sampling" for texture mapping

UV interpolation P_NEAREST P_LINEAR

Texture lookup. For every covered screen-space sample, barycentric weights interpolate the triangle's texture coordinates to obtain a continuous (u, v) position.

Nearest vs. bilinear. Nearest sampling uses the closest texel (one lookup), preserving hard texel boundaries but producing blocky magnification. Bilinear sampling blends the four surrounding texels, giving smoother reconstruction at the cost of four lookups and interpolation.

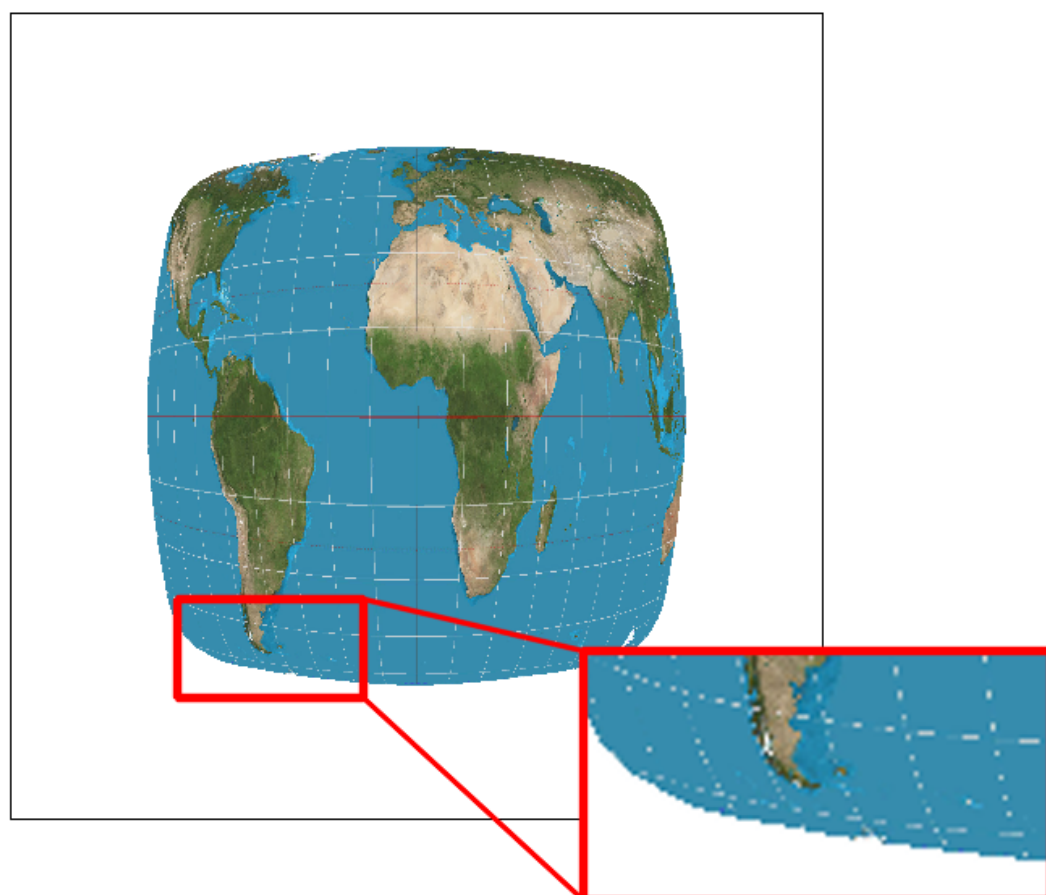


Figure 5.1: Nearest pixel sampling, 1 sample per pixel

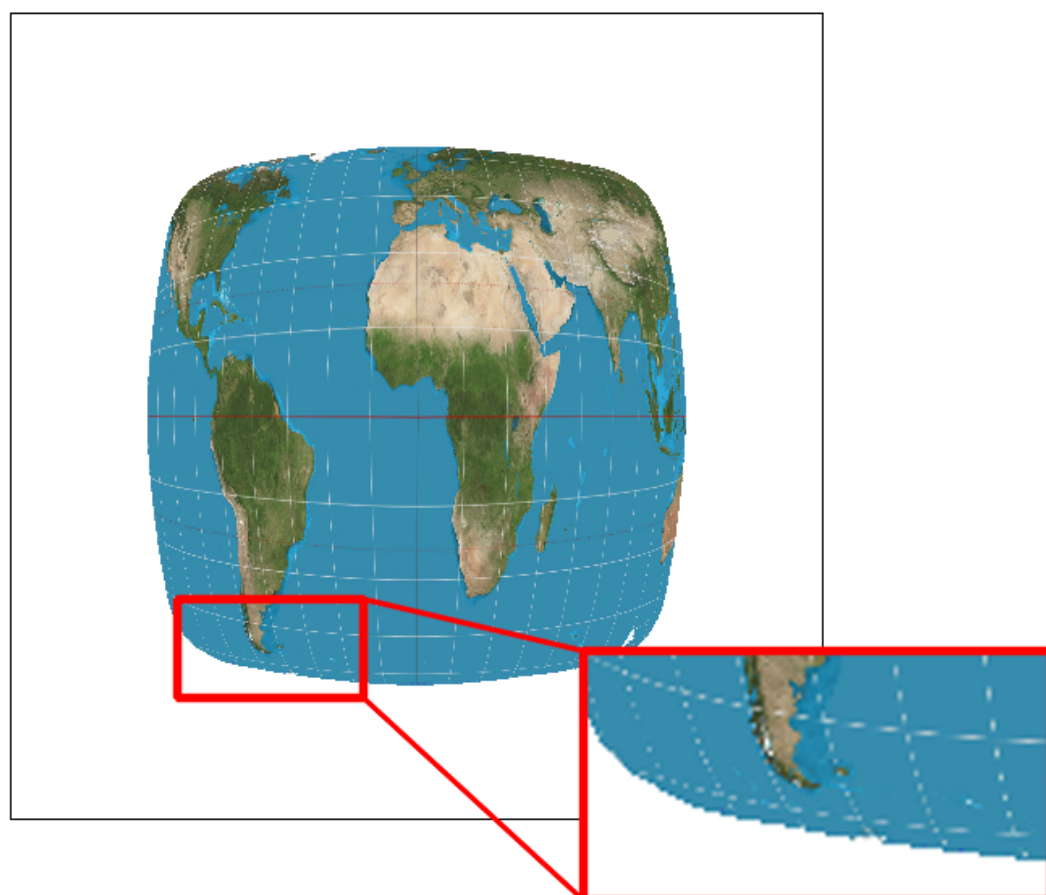


Figure 5.2: Bilinear pixel sampling, 1 sample per pixel

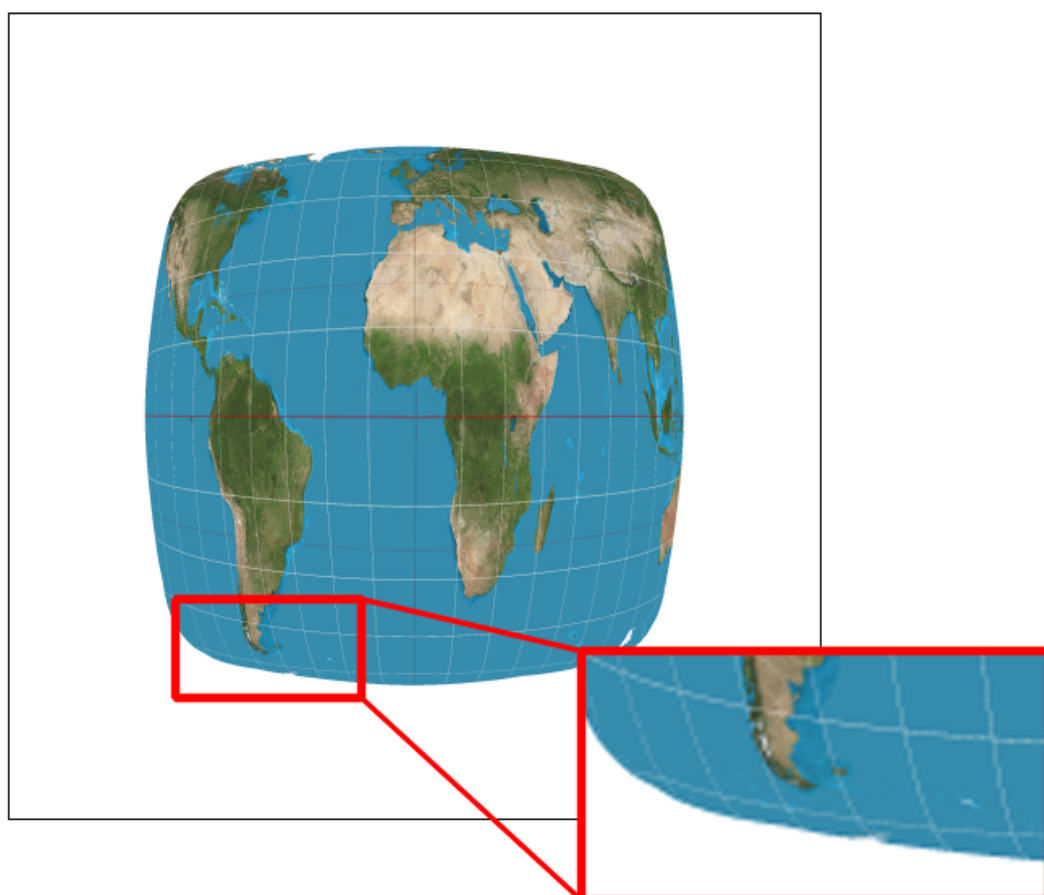


Figure 5.3: Nearest pixel sampling, 16 samples per pixel

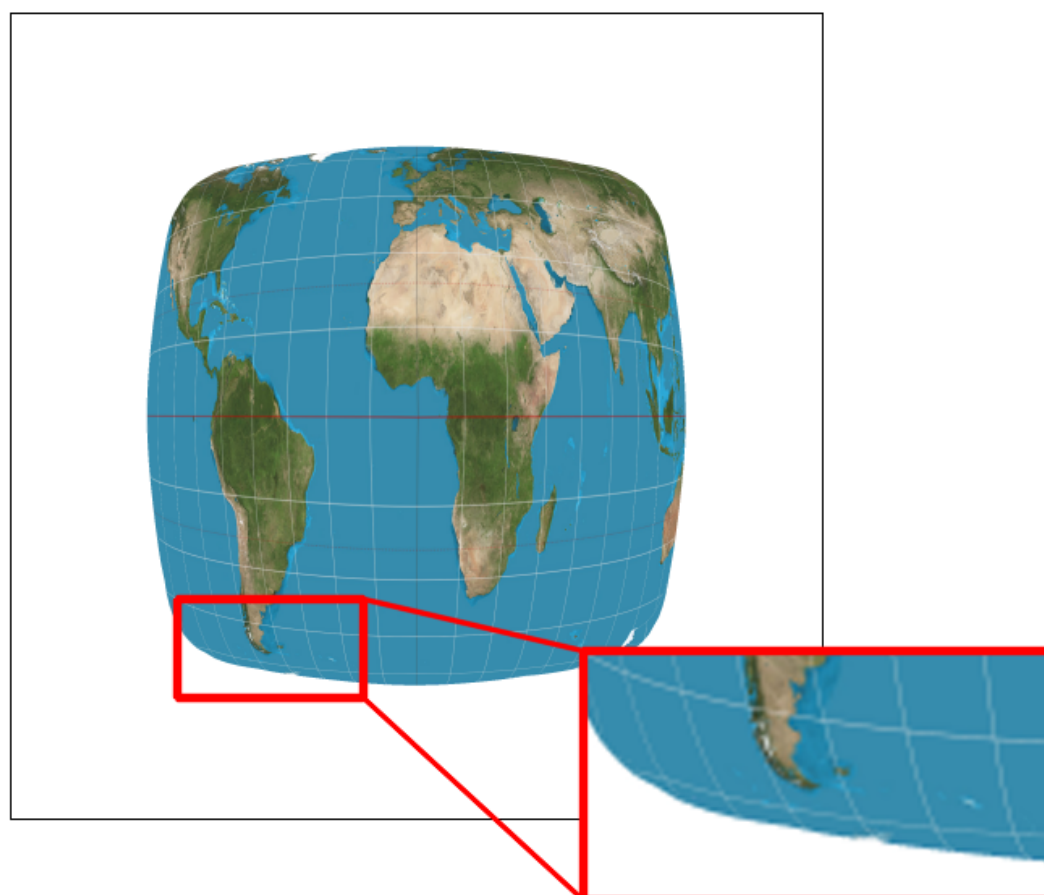


Figure 5.4: Bilinear pixel sampling, 16 samples per pixel

Observation. Nearest sampling shows the strongest stair steps around coastlines and thin grid lines. Bilinear filtering softens those texel transitions, while $16\times$ supersampling mainly improves geometric edge coverage. The nearest-bilinear gap is largest under magnification or near high-contrast, subpixel texture features.

Task 6: "Level Sampling" with mipmaps for texture mapping

Mipmaps `L_ZERO` `L_NEAREST` `L_LINEAR`

Level selection. We interpolate UVs at (x, y) , $(x + 1, y)$, and $(x, y + 1)$. Their finite differences estimate how many base-level texels one screen pixel covers. After scaling by texture width/height, we compute the larger footprint ρ and use $D = \log_2(\max(\rho, 1))$, clamped to the available mip levels.

Sampling modes. `L_ZERO` always uses level 0; `L_NEAREST` rounds D ; `L_LINEAR` blends the two adjacent mip levels. Combining `L_LINEAR` with bilinear pixel sampling gives trilinear filtering.

Custom texture. For this comparison we use a custom spiral PNG, chosen because its tightly compressed center makes mipmap aliasing and filtering differences easy to see.

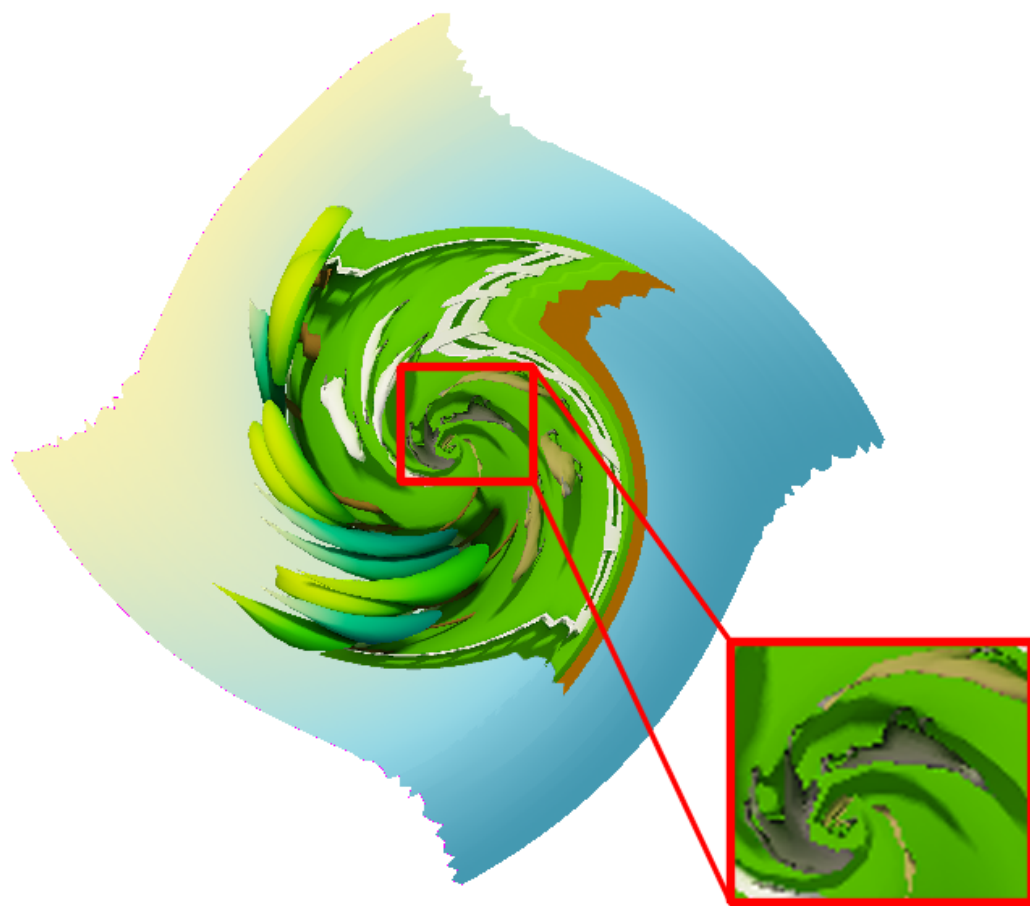


Figure 6.1: Nearest pixel sampling; mipmap level 0

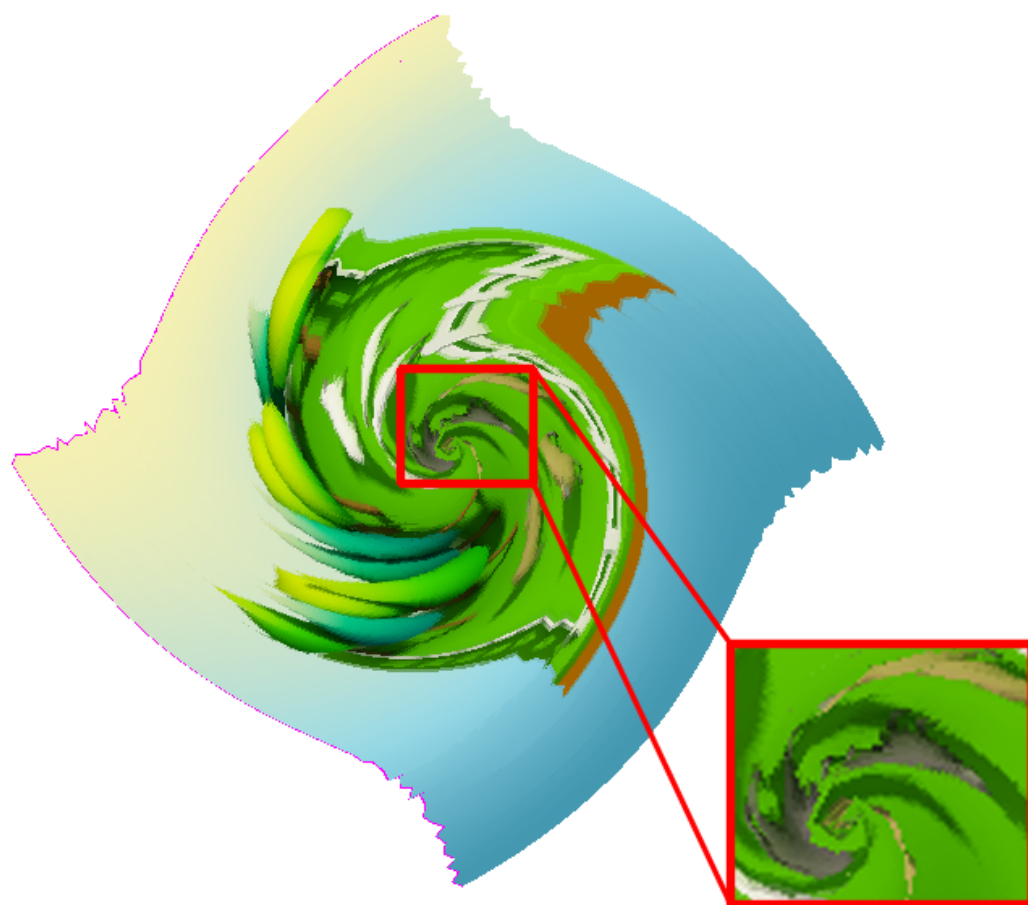


Figure 6.2: Nearest pixel sampling; nearest mipmap level

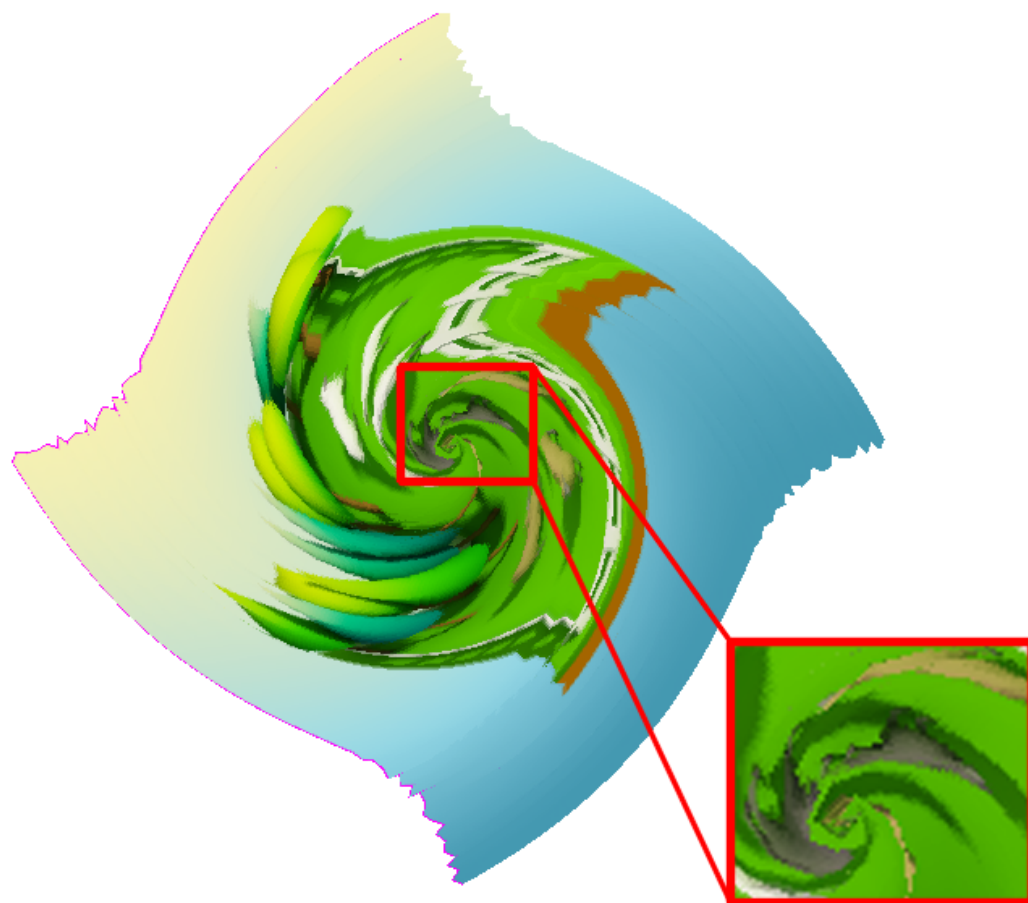


Figure 6.3: Nearest pixel sampling; linear mipmap level blending

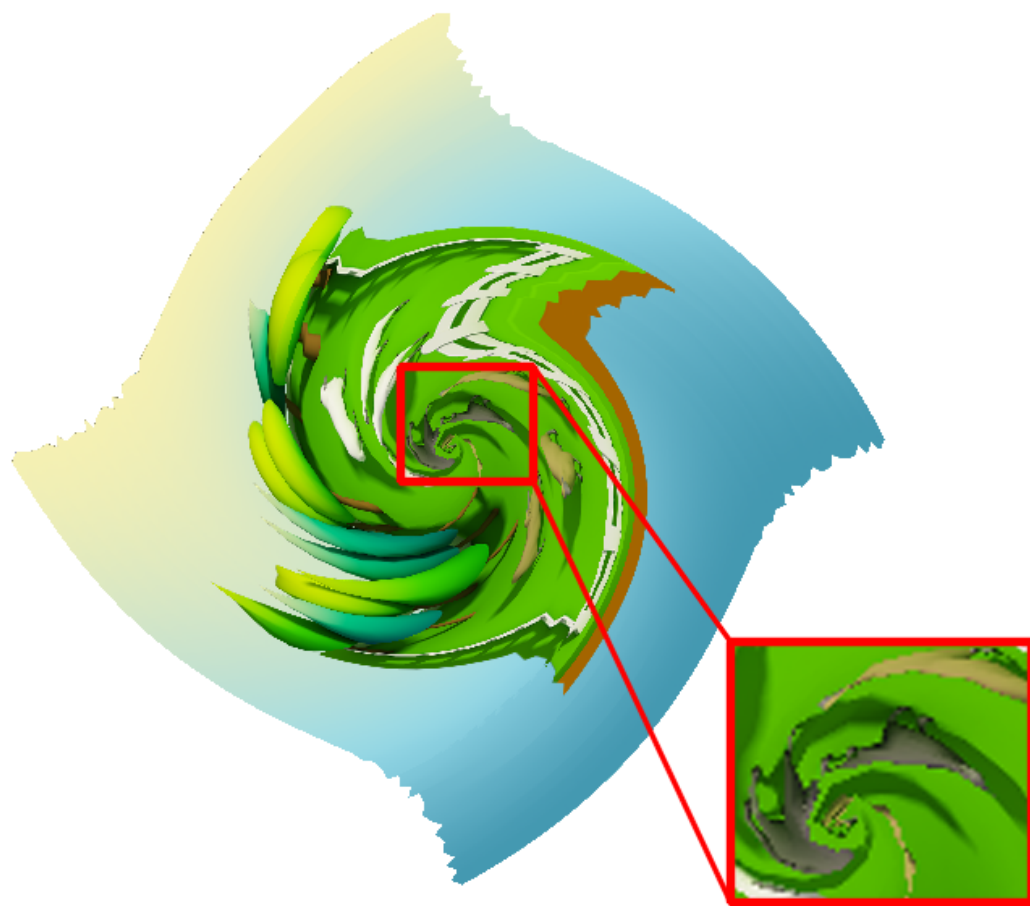


Figure 6.4: Bilinear pixel sampling; mipmap level 0

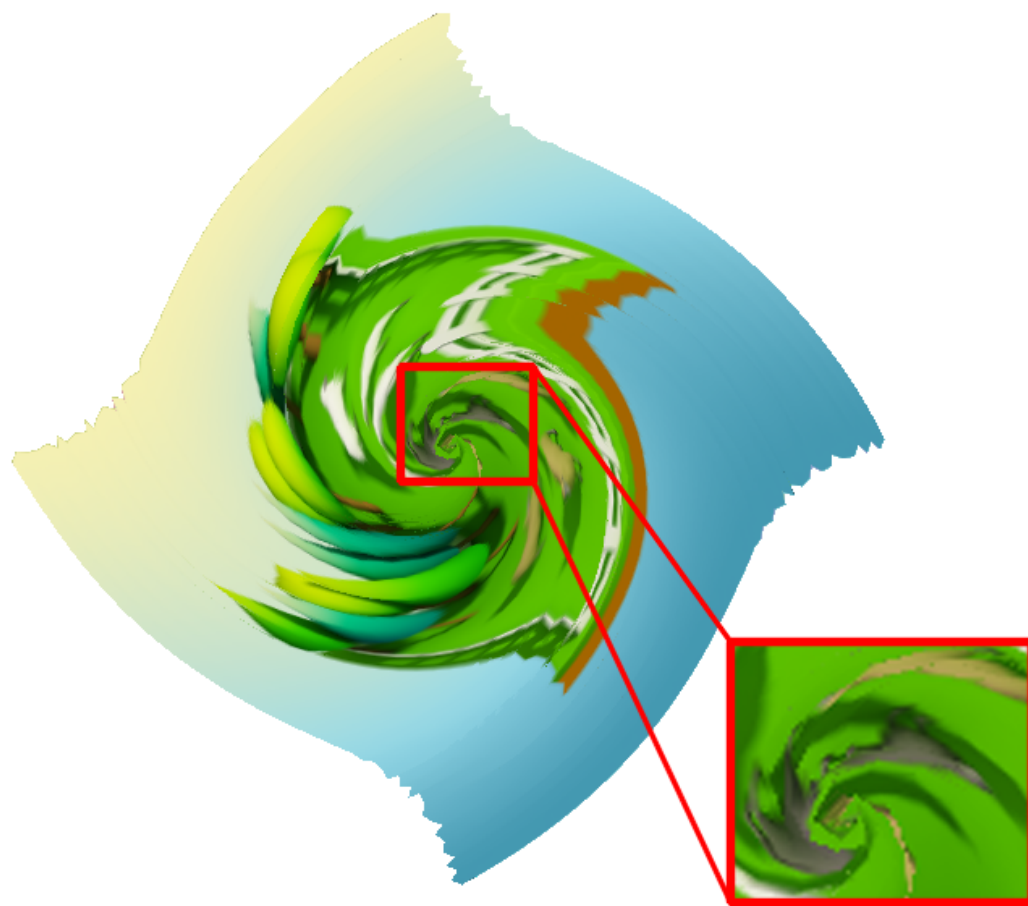


Figure 6.5: Bilinear pixel sampling; nearest mipmap level

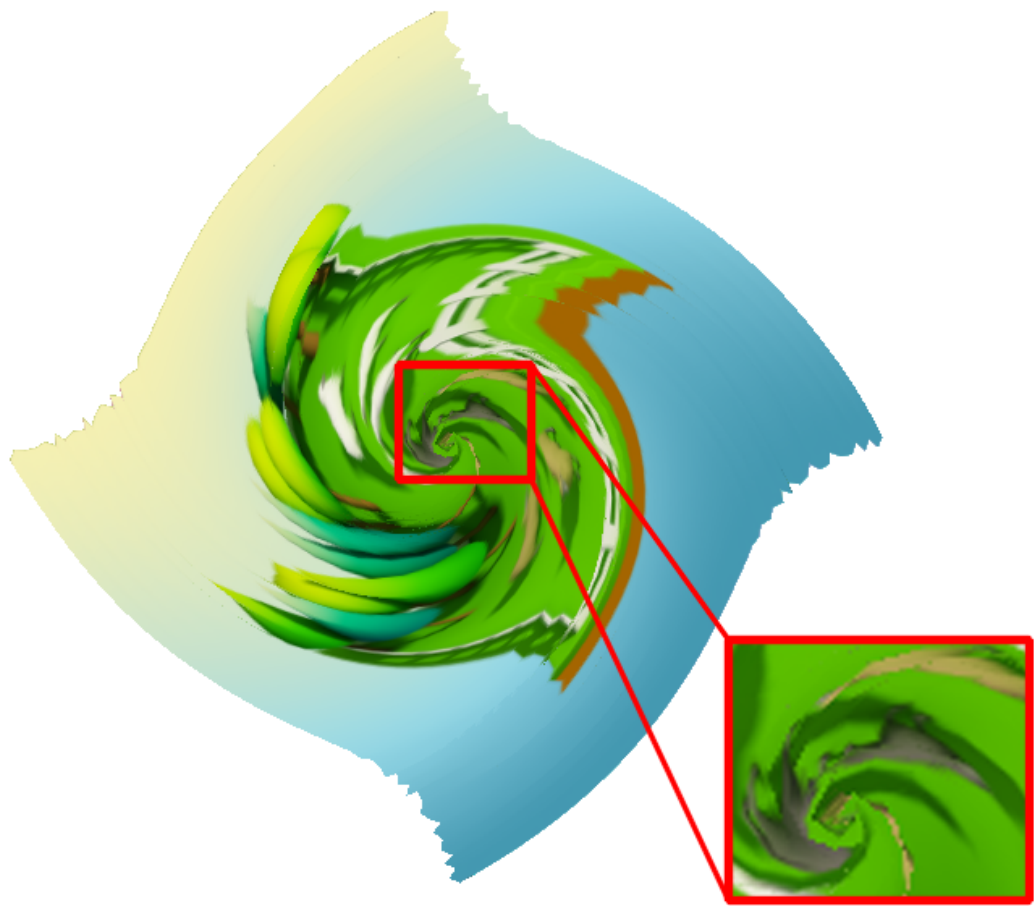


Figure 6.6: Bilinear pixel sampling; linear mipmap level blending

Observation. Level 0 preserves the most source detail but aliases strongly near the compressed spiral center. Nearest-level mipmapping removes much of that minification aliasing; linear level blending further smooths transitions between mip levels. Bilinear pixel sampling smooths texel boundaries within whichever level is selected.

Sampling trade-offs

Technique	Speed	Memory	Antialiasing strength
Pixel sampling	Nearest is cheapest; bilinear performs four texel lookups.	Little additional persistent memory.	Bilinear improves texture reconstruction and magnification.
Level sampling	Mipmap lookup is efficient; linear level blending samples two levels.	The mip pyramid adds roughly one-third extra texture storage.	Very effective for texture minification and high-frequency aliasing.
Supersampling	Cost grows roughly with the sample rate.	Sample-buffer memory also grows with the sample rate.	General-purpose: improves geometric edges and subpixel coverage.